



Programovacie jazyky

Haskell – úvod do jazyka (infraštruktúra, typy, syntax)

RNDr. Richard Ostertág, PhD. (ostertag@dcs.fmph.uniba.sk)

22. februára 2024

Katedra informatiky, Fakulta matematiky, fyziky a informatiky
Univerzita Komenského, Bratislava

Infraštruktúra

Haskell (podľa Haskell B. Curry), 1990, Paul Hudak, John Hughes, Simon Peyton Jones, Erik Meijer, Philip Wadler, ...

- čistý (pure) funkcionálny jazyk, t.j. bez vedľajších efektov
 - monadický V/V (monadic I/O) (až v 1996)
- lenivé (lazy, non-strict) vyhodnocovanie argumentov (vs. strict, eager)
 - modulárne programovanie potrebuje dobré lepidlo
 - efektívnejšie skladanie funkcií: `any p list = or (map p list)`
 - nekonečné dátové štruktúry (`nat = 1 : map (+1) nat`)
 - lenivosť stojí čas a priestor (komplikuje úvahy o zložitosti – najmä priestorovej)
- statická typová kontrola (typy nie je nutné písať, odvodí ich sám – väčšinou)
- zaviedol triedy typov (type classes)
- používa sa dodnes
- ovplyvnil: C#/LINQ, Elm, F#, Java/Generics, ...

Motto: „Avoid success at all costs.“ $\Rightarrow$ umožňuje rýchly vývoj

- Haskell má viacero implementácií (spomenieme len dve)
 - Glasgow Haskell Compiler (GHC)
 - kompiluje do strojového kódu
 - „de facto“ štandardný dialekt
 - Haskell User's Gofer System (Hugs)
 - interpreter (kompiluje do „bytecode“)
 - už sa aktívne nevyvíja, nahradený GHCi (GHC s REPL)
- Hackage (<https://hackage.haskell.org/>)
 - komunitný centrálny archív open source balíčkov ($\approx 16\,000$)
- Cabal (Common Architecture for Building Applications and Libraries)
 - nástroj pre zostavovanie programov a správu balíčkov
 - konflikt vo verziách závislostí $\rightsquigarrow$ „Cabal hell“ („cabal new-build“ mnohé rieši)
- Stack (<https://www.haskellstack.org/>) („stack“ stavia na „cabal“)
 - využíva výber kompatibilných balíčkov: „Stackage“
 - stabilnejší (trošku menej aktuálny) variant „Hackage“-u
 - Haskell LTS (<https://www.stackage.org/lts>)

stiahnuť a spustiť inštaláciu pre Windows 64-bit

(https://www.stackage.org/stack/windows-x86_64-installer)

- potvrdiť preddefinované nastavenia
- nainštaluje sa jediný program „stack.exe“
 - do adresára AppData\Roaming\local\bin
 - pridá tento adresár do cesty
- následne je možné „stack“ spúšťať z príkazového riadku

Alternatívne je možné použiť „novinku“ [GHCup](#).

- otvoriť príkazový riadok (cmd)
- zadať: „stack setup“ ... čakať ... nainštaluje GHC do adresárov:
 - AppData\Local\Programs\stack
 - C:\sr (vrátane podadresárov)
 - globálna konfigurácia sa nachádza v súbore:
C:\sr\global-project\stack.yaml
- zatvoriť a znovu otvoriť príkazový riadok
 - načítanie nových premenných prostredia
- zadať: „stack ghci“
 - ukončenie cez „:q“

Vytvorenie projektu

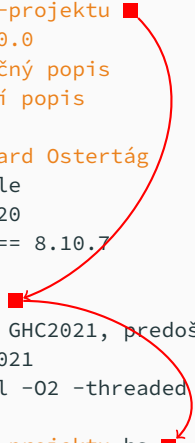
- vytvorenie nového projektu zo šablóny
 - `stack new meno-projektu meno-šablóny`, napríklad:
 - `stack new helloworld new-template`
 - `stack new helloworld franklinchen`
 - `stack new helloworld simple`
 - vytvorí podadresár „meno-projektu“ a v ňom kostru
 - `stack templates` – vráti zoznam dostupných šablón
- stiahnutie a inštalácia správnej verzie GHC
 - `stack setup`
- zostavenie spustiteľného programu
 - `stack build` (pracuje v podadresári „stack-work“)
- spustenie programu
 - `stack exec helloworld-exe`
 - podľa riadku: `executable helloworld-exe (v helloworld.cabal)`

Pozri: <https://docs.haskellstack.org/en/stable/GUIDE/>

Súbor „meno-projektu“.cabal

- nepoužívať tabulátor, iba medzery

```
1  name:                meno-projektu ■
2  version:             0.1.0.0
3  synopsis:            stručný popis
4  description:         dlhší popis
5  license:             BSD3
6  author:              Richard Ostertág
7  build-type:          Simple
8  cabal-version:       >=1.20
9  tested-with:         GHC == 8.10.7
10
11 executable meno-projektu ■
12   -- Najnovšia verzia je GHC2021, predošlá Haskell2010.
13   default-language:  GHC2021
14   ghc-options:       -Wall -O2 -threaded -rtsopts -with-rtsopts=-N
15   hs-source-dirs:    .
16   main-is:           meno-projektu.hs ■
17   build-depends:     base
18                     , containers
```



Súbor stack.yaml

- môžeme si vybrať, aká verzia Stackage sa použije
 - odporúčam poslednú LTS (Long Term Support) – aktuálne 22.11 (s ghc-9.6.4)
 - obsahuje $\approx$ 3333 balíčkov
 - zmena 7.11 $\rightarrow$ 7.12 $\approx$ týždenne
 - API kompatibilné zmeny, nové balíčky
 - zmena 7.19 $\rightarrow$ 8.0 $\approx$ polročne
 - API nekompatibilné zmeny, nové aj odstránené balíčky
- netreba nič meniť (iba resolver, ak chceme novší)

```
1 resolver: lts-22.11
2 packages:
3 - '.'
4 extra-deps: []
5 flags: {}
6 extra-package-dbs: []
```

- netreba nič meniť
- môžeme úplne vynechať, pokiaľ v cabal súbore je:
 - build-type: Simple

```
1 import Distribution.Simple
2 main = defaultMain
```

Súbor „meno-projektu“.hs

```
1 module Main (main) where
2
3 import Data.List (sort)
4 import Data.Set (Set, singleton, member, insert)
5
6 -- kód modulu
7
8 main :: IO ()
9 main = putStrLn "Hello, World!" -- vykonanie hlavnej činnosti
```

- stack install hlint ... čakať ...
 - stiahne a skompiluje potrebné balíčky
 - nainštaluje jediný program „hlint.exe“
 - do adresára AppData\Roaming\local\bin
 - adresár je už pridaný do cesty
- hlint „meno-projektu“.hs
 - vypíše návrhy pre zlepšenie kódu v „meno-projektu“.hs

Užitočné linky

Haskell online, okamžite, bez inštalácie: <https://tryhaskell.org/>

IRC kanál, kde sa dá nájsť pomoc:

- <https://web.libera.chat/>
 - kanál “haskell”
- nie len od ľudí, ale aj od robota:
 - `/query lambdabot`
 - `@p! \x y -> y:x ~> flip (:)`
 - ďalšie info. na <https://wiki.haskell.org/Lambdabot>

Nájsť štandardnú funkciu podľa mena alebo typu je možné na:

- Hoogle (<https://www.haskell.org/hoogle/>)

Haskell 2010 Language Report:

- <https://www.haskell.org/onlinereport/haskell2010/>

Práca s GHCi

Interné príkazy GHCi

- :? alebo :h – nápoveda
- :t <expr> – ukáže typ <expr>; :i <name> – zobrazí informácie o <name>
- :l <module> – nahrá <module>; :r – znovu načíta aktuálne moduly
- :ma – spustí main funkciu
- Ctrl+l – vymaže obrazovku; Ctrl+d – to isté ako :q – koniec
- :set <option> – nastaví <option> (opak je :unset <option>)
- :set -XRecordWildCards – aktivuje rozšírenie jazyka
 - +s – spotrebovaný čas a priestor
 - +t – typ výsledku

```
1 Prelude> :set +s +t
2 Prelude> length [1..1000000000]
3 1000000000
4 it :: Int
5 (1.81 secs, 7,200,312,064 bytes)
```

Lenivý výpis

- `:p <name>` – ukáže hodnotu `<name>` bez vynútenia vyhodnotenia
- `:sp <name>` – zjednodušene ukáže hodnotu `<name>` bez vynútenia vyhodnotenia

```
1 Prelude> x = replicate 10 (length [1..10])
2 Prelude> :p x
3 x = (_t1::[Int])
4 Prelude> y = length x
5 Prelude> :p y
6 y = (_t2::Int)
7 Prelude> y
8 10
9 Prelude> :p y
10 y = 10
11 Prelude> :p x
12 x = [(_t3::Int),(_t4::Int), ... ,(_t12::Int)]
13 Prelude> :sp x
14 x = [_,-,-,-,-,-,-,-,-,-,-]
```


Viacriadkový vstup

```
1 Prelude> :{  
2 Prelude| fact 0 = 1  
3 Prelude| fact n = n * fact (n - 1)  
4 Prelude| :}  
5 Prelude> fact 10  
6 3628800
```

```
1 Prelude> let {fact 0 = 1; fact n = n * fact (n - 1)}  
2 Prelude> fact 10  
3 3628800
```

```
1 Prelude> {fact 0 = 1; fact n = n * fact (n - 1)}  
2 Prelude> fact 10  
3 3628800
```

Základné typy

Vybrané nečíselné typy

- Logické hodnoty: **data Bool = False | True**
- Znaky: **data Char** $\approx$ 'a' | 'A' | 'b' | 'B' | ...
- Zoznamy: **data []** a = [] | a : [a]
 - 'a' : 'h' : 'o' : 'j' : [] $\equiv$ ['a', 'h', 'o', 'j']
 - homogénne (všetky prvky sú rovnakého typu)
 - premenlivej dĺžky (potencionálne ∞)
- Reťazce: **type String = [Char]**
 - "ahoj" $\equiv$ ['a', 'h', 'o', 'j']
- N-tice: (a,b,c)
 - (10, **True**, "ahoj")
 - heterogénne (môžu spájať rôzny typy)
 - fixnej dĺžky (vždy konečné)
- Unit: **data () = ()** (niečo iné ako **Data.Void**)
 - typ () má iba jedinú hodnotu a tou je ()

Základné číselné typy

- **Integral** (div, mod, ...)
 - **Integer**: celé čísla ľubovoľnej veľkosti (aka BigInt)
 - **Int**: celé čísla pevnej presnosti (8 bajtov)
 $\langle -9223372036854775808, 9223372036854775807 \rangle$
 - **Word**: nezáporné celé čísla pevnej presnosti (8 bajtov) $\langle 0, 18446744073709551615 \rangle$
- **Ratio** a: racionálne čísla nad **a** (denominator (10 % 4) $\rightsquigarrow$ 2)
 - **import Data.Ratio**
 - $10 \% 4 \rightsquigarrow 5 \% 2$
- **RealFloat** (pi, log, sin, round, floor, isInfinite (1.2 / 0), ...)
 - **Float**: reálne čísla jednoduchá presnosť (4 bajty)
 - **Double**: reálne čísla dvojité presnosť (8 bajtov)
- **Complex** a: komplexné čísla nad **a**
 - **import Data.Complex**
 - $(0 \text{ :+ } 1) * (0 \text{ :+ } 1) \rightsquigarrow (-1.0) \text{ :+ } 0.0$

Základné syntaktické konštrukcie

Identifikátory

- začínajú písmenom
- môžu obsahovať písmená, cifry, podčiarkovník `_` a apostrof `'`
 - záleží na veľkosti písmen
- sú rozdelené do dvoch menných priestorov:
 - identifikátory pre premenné – začínajú malým písmenom
 - identifikátory pre konštruktory – začínajú veľkým písmenom
- identifikátor `_`
 - sa používa ako „don't care“ znak pri porovnávaní vzorov
 - neviaže na seba hodnotu, dá sa použiť opakovane
- identifikátor začínajúci na `_` (napr. `_meno`)
 - viaže hodnotu, nedá sa použiť opakovane v jednom vzore
 - potláča upozornenie na nepoužitý identifikátor
- rezervované identifikátory:
 - case, class, data, default, deriving, do, else, foreign, if, import, in, infix, infixl, infixr, instance, let, module, newtype, of, then, type, where, _

Symboly operátorov

- Haskell umožňuje definovať vlastné binárne operátory
- pozostávajú z aspoň jedného z nasledovných symbolov:
 - `!, #, $, %, &, *, +, ., /, <, =, >, ?, @, \, ^, |, -, ~, :`
- sú rozdelené do dvoch menných priestorov:
 - infix dátové konštruktory – operátor začínajúci symbolom `:`
 - infix binárne funkcie – začínajú iným symbolom
- rezervované operátory:
 - `.., ::, :::, =, \, , |, <-, ->, @, ~, =>`

Komentáre

- sú považované za biely priestor (whitespace)
- sú dvoch druhov:
 - postupnosť dvoch a viac po sebe idúcich pomlčiek
 - postupnosť nesmie byť časťou platného operátora (napr. --> alebo |--, ale --foo začína komentár)
 - **let** a = 42 -- komentár
 - komentár pokračuje až do konca riadku
 - {- ... -} – viacriadkové vnoriteľné komentáre

```
1 {- komentár môže mať  
2 aj {--- vnorené -} komentáre --}  
3 let a = 42 -- komentár
```

- používajú sa aj pre direktívy pre kompilátor (compiler pragmas)
{-# direktíva #-}, napríklad zapnutie všetkých upozornení:
{-# OPTIONS_GHC -Wall #-}

Odsadzovanie kódu

1. Ak po kľúčových slovách **where**, **let**, **do**, **of** nenasleduje {, tak sa zapamätá stĺpec nasledujúcej lexémy (môže byť aj na tom istom riadku) a vloží sa pred ňu { – začiatok zoznamu položiek
 - ak je toto odsadenie menšie ako aktuálne, tak sa zoznam ukončí }, zabudneme na toto odsadenie a pokračujeme
2. na nasledujúcich riadkoch, ak sú odsadené:
 - viac (alebo obsahujú iba biele znaky), tak ide o pokračovanie predošlej položky, nič sa nevkladá
 - rovnako, tak ide o ďalšiu položku, na začiatok riadku sa vloží ;
 - menej, tak ide o ukončenie zoznamu, na začiatok sa vloží }
 - aktuálne odsadenie zmenšíme o úroveň a pokračujeme
3. aktuálny zoznam sa ukončí vložením } pred nasledujúcu lexému, ak je táto neplatná v danom kontexte, ale mohla by ju nahradiť }
 - aktuálne odsadenie zmenšíme o jednu úroveň a pokračujeme

Odsadzowanie kódu

```
1 module AStack(Stack, push, pop, top, size) where
2     data Stack a = Empty
3                 | MkStack a (Stack a) deriving Show
4
5     push :: a -> Stack a -> Stack a
6     push x s = MkStack x s
7
8     size :: Stack a -> Int
9     size s = length (stkToLst s) where
10         stkToLst Empty          = []
11         stkToLst (MkStack x s) = x:xs where xs = stkToLst s
12
13     pop :: Stack a -> (a, Stack a)
14     pop (MkStack x s)
15         = (x, case s of r -> i r where i x = x )
16
17     top :: Stack a -> a
18     top (MkStack x s) = x
19     -- koniec
```

Odsadzovanie kódu – doplnené zátvorky

```
1  module AStack(Stack, push, pop, top, size) where
2      {data Stack a = Empty
3          | MkStack a (Stack a) deriving Show
4
5      ;push :: a -> Stack a -> Stack a
6      ;push x s = MkStack x s
7
8      ;size :: Stack a -> Int
9      ;size s = length (stkToLst s) where
10         {stkToLst Empty          = []
11         ;stkToLst (MkStack x s) = x:xs where {xs = stkToLst s
12
13     }};pop :: Stack a -> (a, Stack a)
14     ;pop (MkStack x s)
15         = (x, case s of {r -> i r where {i x = x}})
16
17     ;top :: Stack a -> a
18     ;top (MkStack x s) = x
19 }
```

-- koniec

Práca so základnými typmi a operátormi

Precedencia operátorov

Pr.	Zľava asociatívne	Neasociatívne	Sprava asoc.
9	!!		.
8			^, ^^, **
7	*, /, 'div', 'mod', 'rem', 'quot'		
6	+, -		
5			:, ++
4		==, / =, <, <=, >, >=, 'elem', 'notElem'	
3			&&
2			
1	>>, >>=		
0			\$, \$!, 'seq'

Precedencia operátorov

- definovateľná precedencia a asociativita vlastných operátorov
 - infixl (zľava asociatívne), infix (neasociatívne), infixr (sprava asociatívne)
 - precedencia: 0 – viaže najslabšie, 9 – najsilnejšie
 - operátor bez deklarácie precedencie má „infixl 9“

```
1 infixl 4 #  
2 x # y = x * x + y * y
```

Použitie operátorov

Sekcie (rezy) operátorov:

- $(\# e) \equiv \backslash x \rightarrow x \# e$
 - legálne iba ak $(x \# e)$ sa parsuje rovnako ako $(x \# (e))$
 - Zle: $(*a+b)$, $(+a+b)$
 - Dobre: $(*(a+b))$, $(+a*b)$, $(+(a+b))$
 - $(-e)$ sa interpretuje ako unárne mínus
 - treba použiť `(subtract e)`
- $(e \#) \equiv \backslash x \rightarrow e \# x$
 - legálne iba ak $(e \# x)$ sa parsuje rovnako ako $((e) \# x)$
 - príklad: `('t')` – odsadzovacia funkcia

Infixová a prefixová forma binárnych operátorov a funkcií:

- operátory sú štandardne infixové: $(1 + 2) \equiv ((+) 1 2)$
- funkcie sú štandardne prefixové (``` – backtick)
 - $(f x y) \equiv (x `f` y)$
 - $1 `f` 2 `f` 3 \equiv (1 `f` 2) `f` 3 \equiv f (f 1 2) 3$

Práca s logickými hodnotami

- **not** – negácia (**not True** $\rightsquigarrow$ **False**)
- **&&** (infixr 3) – konjunkcia (a zároveň)
 - skrátané vyhodnocovanie
 - pri lenivom vyhodnocovaní argumentov netreba skrátané vyhodnocovanie zahrnúť ako výnimku do špecifikácie jazyka
- **||** (infixr 2) – disjunkcia (alebo)
 - skrátané vyhodnocovanie
- **==** (infix 4) – test na rovnosť
 - **1==1==1** $\rightsquigarrow$ Precedence parsing error
 - cannot mix '=' [infix 4] and '=' [infix 4] in the same infix exp.
- **0 || True** $\rightsquigarrow$ typová chyba

Práca s celými číslami

- $+$, $-$ (infixl 6) – sčítanie, odčítanie
- **negate** alebo „-“ – unárne mínus (záporné znamienko)
 - $2 + -3 \rightsquigarrow$ Precedence parsing error, cannot mix
 - $'+'$ [infixl 6] and prefix $'-'$ [infixl 6] in the same infix exp.
 - správne: $2 + (-3)$ alebo $2 + \text{negate } 3$
- $*$ (infixl 7) – násobenie
- $2^3 \rightsquigarrow 8$ (Num), $2^{(-3)} \rightsquigarrow 0.125$ (Fractional), $2**3 \rightsquigarrow 8.0$ (Floating)
 - infixr 8
- $/=$ (infix 4) – test na nerovnosť
 - $1/=2 \rightsquigarrow$ **True**

Usporiadané n -tice

- `(1,2,True,"Yoda")` – štvorica (n je neobmedzené, minimálne do 15)
 - `()` – Unit – dá sa chápať aj ako n -tica pre $n = 0$
- `fst :: (a, b) -> a` – vráti 1. prvok dvojice
 - `fst (21, True) ~> 21`
- `snd :: (a, b) -> b` – vráti 2. prvok dvojice
 - `snd (21, True) ~> True`
- `swap :: (a, b) -> (b, a)` – vymení prvky dvojice
 - `import Data.Tuple`
 - `swap (21, True) ~> (True,21)`

- `[]` – „nil“ – prázdny zoznam
- `:` – „cons“ – `a:list` pridá `a` na začiatok list (v čase $O(1)$)
 - zvykne sa písať `head:tail`, `h:t` alebo `x:xs`
 - `10:[]` $\rightsquigarrow$ `[10]`, `10:[7,4,9]` $\rightsquigarrow$ `[10,7,4,9]`
- `head :: [a] -> a` – vráti 1. prvok zoznamu (v čase $O(1)$)
- `tail :: [a] -> [a]` – vráti chvost zoznamu (v čase $O(1)$)
- `init :: [a] -> [a]` – vynechá posledný prvok ($O(n)$)
- `last :: [a] -> a` – vráti posledný prvok zoznamu ($O(n)$)
 - tieto funkcie nie sú „bezpečné“ (zavolanie na `[]` $\rightsquigarrow$ Exception: empty list)
- `(++) :: [a] -> [a] -> [a]` (infixr 5) – spojí 2 zoznamy ($O(n)$)
- `null :: [a] -> Bool` – je zoznam prázdny? (v čase $O(1)$)
- `length :: [a] -> Int` – dĺžka zoznamu (v čase $O(n)$)
- `(!!) :: [a] -> Int -> a` (infixl 9) – vráti i -ty prvok
 - `[10,11,12]!!2` $\rightsquigarrow$ `12` (indexuje od 0, v čase $O(n)$)
 - `[10,11,12]!!3` $\rightsquigarrow$ Exception: index too large

Aritmetické postupnosti

Ak `e1`, `e2` a `e3` sú typu `t`, ktorý je inštanciou triedy **Enum**, tak:

- `[e1 ..]` $\equiv$ `enumFrom e1`
- `[e1,e2 ..]` $\equiv$ `enumFromThen e1 e2`
- `[e1 .. e3]` $\equiv$ `enumFromTo e1 e3`
- `[e1, e2 .. e3]` $\equiv$ `enumFromThenTo e1 e2 e3`

t.j. `[ .. ]` je syntaktický cukor, umožňujúci elegantne zapísať zoznam, obsahujúci prvky aritmetickej postupnosti:

- `[1..5]` $\rightsquigarrow$ `[1,2,3,4,5]`
- `[1,3..]` $\rightsquigarrow$ `[1,3,5,7,9,11,13,15,17,19,21,...]`
- `[False ..]` $\rightsquigarrow$ `[False,True]`

Množinová notácia (list comprehensions)

má tvar $[e | q_1, \dots, q_n], n \geq 1$, kde kvalifikátor q_i je:

- *generátor* tvaru $p \leftarrow e$, kde p je vzor typu t a e je výraz typu $[t]$
- **let** var $=$ exp, pričom **var** je použiteľné v e a nasledujúcich kvalifikátoroch
- *stráž* (boolean guard), ľubovoľný výraz typu **Bool**

Príklady:

- $[(y, x2) \mid x \leftarrow [1..9], \text{let } x2 = x*x, y \leftarrow [-2, -3], x2 < 5] \rightsquigarrow [(-2,1), (-3,1), (-2,4), (-3,4)]$
- $[x \mid xs \leftarrow [[(1,2), (3,4)], [(3,2)]], (3,x) \leftarrow xs] \rightsquigarrow [4,2]$
- $[x \mid x \leftarrow x, x \leftarrow x] \equiv [z \mid y \leftarrow x, z \leftarrow y]$

Základná práca so zoznamami

- `map :: (a -> b) -> [a] -> [b]`
 - `map f [x1, x2, ...] ~> [f x1, f x2, ...]`
 - `map (+10) [1..5] ~> [11,12,13,14,15]`
- `reverse :: [a] -> [a]` – otočenie zoznamu
- `transpose :: [[a]] -> [[a]]` – tansponovanie matice
 - `transpose [[1,2,3],[4,5,6]] ~> [[1,4],[2,5],[3,6]]`

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}^T$$

- `zip :: [a] -> [b] -> [(a, b)]`
 - `zip [1,2,3] ['a','b'] ~> [(1,'a'),(2,'b')]`